

66. Un texte qui porte un nom

Le même message à deux endroits, écrit une fois.

1. Ce qu'il faut comprendre

`const char NOM[] = "..."` est gravé dans la cartouche, comme une table. Il s'écrit ensuite **partout où** `texte()` attend des guillemets.

Le même message répété trois fois entre guillemets, ce sont trois occasions de le changer à deux endroits. Ici il n'y en a qu'un.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **372 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         29 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   85 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles        112 o ← CopierTuiles - 7 lettres sur 43
"GAME OVER"    9 o ← texte() - un texte gravé
```

Lettres gravées : AEGMORV

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 2    22 o sur place, dans main() + EcrireTexte (10 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : écrit deux fois le même texte, grâce à un NOM.
//
// const char TITRE[] = "GAME OVER";
// const char ...[] = un texte gravé dans la cartouche, qui ne change pas.
// On l'écrit une fois, et on s'en sert partout par son nom.

#include <texte> // écrit un texte à l'écran
```

2. UN TEXTE NOMMÉ — TITRE

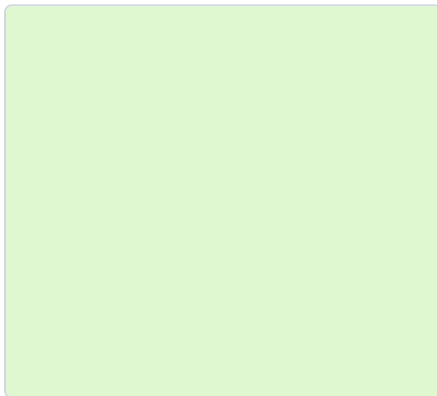
```
const char TITRE[] = "GAME OVER"; // Le texte, rangé sous le nom TITRE.
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

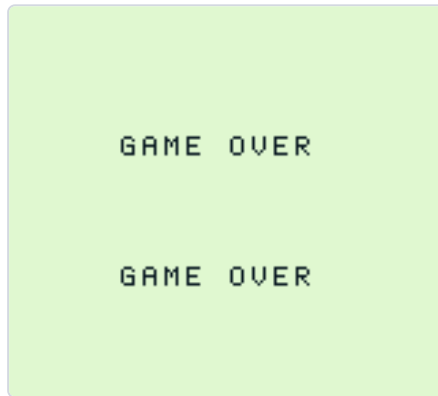
```
int main() { // Le jeu commence ici.  
    texte(5, 6, TITRE); // GAME OVER, ligne 6...  
    texte(5, 12, TITRE); // ... et encore, ligne 12. Pour le changer : une seule ligne.  
  
    while (true) { // Répète pour toujours :  
        image(); // attend l'image suivante.  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

« GAME OVER » deux fois, écrit une seule fois dans le programme.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le message est écrit en haut

✓ et en bas

✓ ce sont les mêmes tuiles

Ensuite : **66.1. Chapitre 10 — Les tableaux : ranger beaucoup de valeurs : ce qui vient** — chapitre 10 : Les tableaux : ranger beaucoup de valeurs. · [gameboy3](#) — écrire une cartouche Game Boy en C++